

```
import seaborn as sns
import pandas as pd
import matplotlib.pyplot as plt

iris = sns.load_dataset("iris")
df= pd.DataFrame(iris)

df
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

```
df.tail()
```

	sepal_length	sepal_width	petal_length	petal_width	species
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

```
df.shape
```

```
(150, 5)
```

```
df.columns
```

```
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
      'species'],
      dtype='object')
```

```
df.isnull().sum()
```

```

      0
sepal_length  0
sepal_width   0
petal_length  0
petal_width   0
species       0

dtype: int64
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal_length    150 non-null   float64
1   sepal_width     150 non-null   float64
```

```
2  petal_length  150 non-null  float64
3  petal_width   150 non-null  float64
4  species       150 non-null  object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
df.dtypes
```

```
      0
sepal_length  float64
sepal_width   float64
petal_length  float64
petal_width   float64
species       object
dtype: object
```

```
df.describe()
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
df.median(numeric_only=True)
```

```
      0
sepal_length  5.80
sepal_width   3.00
petal_length  4.35
petal_width   1.30
dtype: float64
```

```
df.min(numeric_only=True)
```

```
      0
sepal_length  4.3
sepal_width   2.0
petal_length  1.0
petal_width   0.1
dtype: float64
```

```
df.std(numeric_only=True)
```

```
      0
sepal_length  0.828066
sepal_width   0.435866
petal_length  1.765298
petal_width   0.762238
dtype: float64
```

```
df.corr(numeric_only=True)
```

	sepal_length	sepal_width	petal_length	petal_width
sepal_length	1.000000	-0.117570	0.871754	0.817941
sepal_width	-0.117570	1.000000	-0.428440	-0.366126
petal_length	0.871754	-0.428440	1.000000	0.962865
petal_width	0.817941	-0.366126	0.962865	1.000000

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["species_encoded"] = le.fit_transform(df["species"])
print(df.head())
print(df.tail())
```

```

   sepal_length  sepal_width  petal_length  petal_width  species \
0           5.1         3.5         1.4         0.2  setosa
1           4.9         3.0         1.4         0.2  setosa
2           4.7         3.2         1.3         0.2  setosa
3           4.6         3.1         1.5         0.2  setosa
4           5.0         3.6         1.4         0.2  setosa

   species_encoded
0                0
1                0
2                0
3                0
4                0

   sepal_length  sepal_width  petal_length  petal_width  species \
145           6.7         3.0         5.2         2.3  virginica
146           6.3         2.5         5.0         1.9  virginica
147           6.5         3.0         5.2         2.0  virginica
148           6.2         3.4         5.4         2.3  virginica
149           5.9         3.0         5.1         1.8  virginica

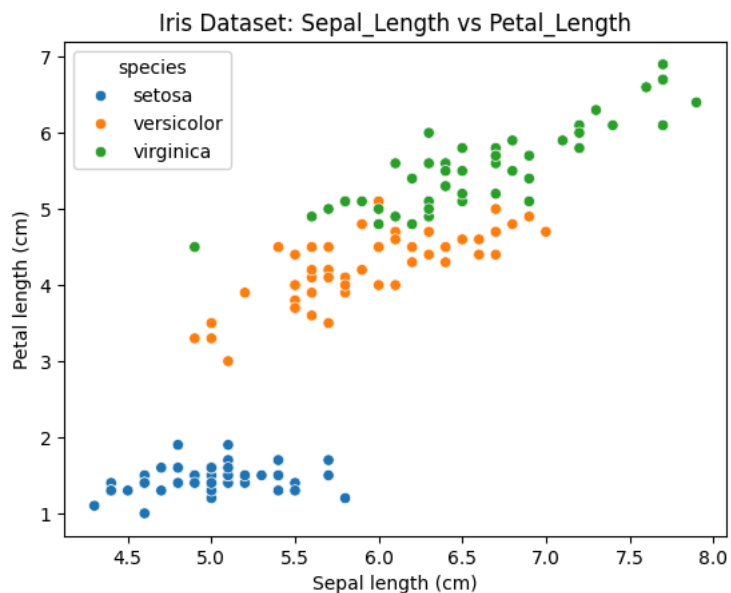
   species_encoded
145                2
146                2
147                2
148                2
149                2
```

```

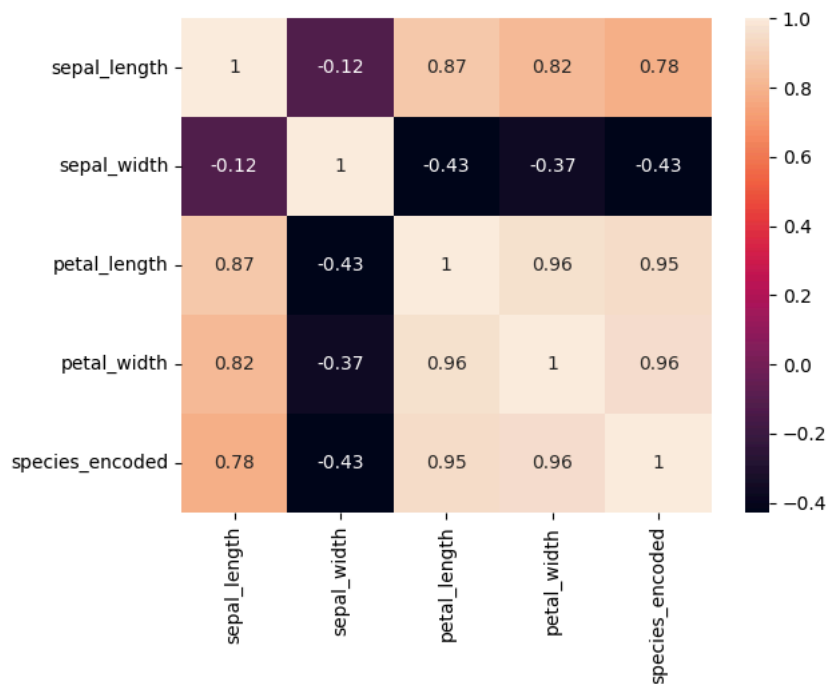
sns.scatterplot(
    data = iris,
    x="sepal_length",
    y="petal_length",
    hue= "species"
)

plt.title("Iris Dataset: Sepal_Length vs Petal_Length")
plt.xlabel("Sepal length (cm)")
plt.ylabel("Petal length (cm)")

plt.show()
```



```
import matplotlib.pyplot as plt
import seaborn as sns
correlation = df.corr()
sns.heatmap(correlation, annot=True)
plt.show()
```



```
x = df.drop(["species", "species_encoded"], axis=1)
y = df["species_encoded"]
```

```
print("Features (X):")
print(x.head())
```

```
print("\n Target (y):")
print(y.head())
```

```
Features (X):
   sepal_length  sepal_width  petal_length  petal_width
0           5.1           3.5           1.4           0.2
1           4.9           3.0           1.4           0.2
2           4.7           3.2           1.3           0.2
3           4.6           3.1           1.5           0.2
4           5.0           3.6           1.4           0.2
```

```
Target (y):
```

```
0    0
1    0
2    0
3    0
4    0
```

```
Name: species_encoded, dtype: int64
```

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)
```

```
print("X_train:", x_train.shape)
print("X_test:", x_test.shape)
print("y_train:", y_train.shape)
print("y_test:", y_test.shape)
```

```
X_train: (120, 4)
X_test: (30, 4)
y_train: (120,)
y_test: (30,)
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
model = KNeighborsClassifier(n_neighbors=5)
model.fit(x_train, y_train)
```

▼ KNeighborsClassifier ⓘ ?
KNeighborsClassifier()

```
y_pred = model.predict(x_test)
```

```
print("Predicted Values:")  
print(y_pred)
```

Predicted Values:
[1 0 2 1 1 0 1 2 1 1 2 0 0 0 1 2 1 1 2 0 2 0 2 2 2 2 2 0 0]

```
new_data = [[5.1, 3.5, 1.4, 0.2]]  
prediction = model.predict(new_data)
```

```
print("Predicted Species code:", prediction[0])
```

Predicted Species code: 0
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but KNeighborsClassifier was fitted with feature names
warnings.warn(

```
from sklearn.metrics import accuracy_score  
accuracy = accuracy_score(y_test, y_pred)  
print("Accuracy:", accuracy)
```

Accuracy: 1.0

```
from sklearn.metrics import confusion_matrix  
cm= confusion_matrix(y_test, y_pred)  
print("Confusion Matrix:")  
print(cm)
```

Confusion Matrix:
[[10 0 0]
[0 9 0]
[0 0 11]]

```
from sklearn.metrics import classification_report  
print("Classification Report:")  
print(classification_report(y_test, y_pred))
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30